# 1 Experimental Setup

The 15-image and 30-image subsets were reconstructed with both exhaustive and sequential matching. The primary camera model was `SIMPLE_PINHOLE`. For Gaussian Splatting, the 30-image exhaustive reconstruction was selected because it registered all 30 images and produced the largest sparse point cloud among the tested configurations.
The GS model used all 30 images during COLMAP reconstruction. Images 10, 20, and 30 were then withheld from GS optimization:
The other 27 views formed the training split. One GS model was optimized to 6000 iterations, with models and checkpoints saved at iterations 2000 and 6000.

# 2 COLMAP Structure from Motion

## 2.1 Quantitative results

Table 1 summarizes the four COLMAP reconstructions. Every input image was successfully registered in every configuration.

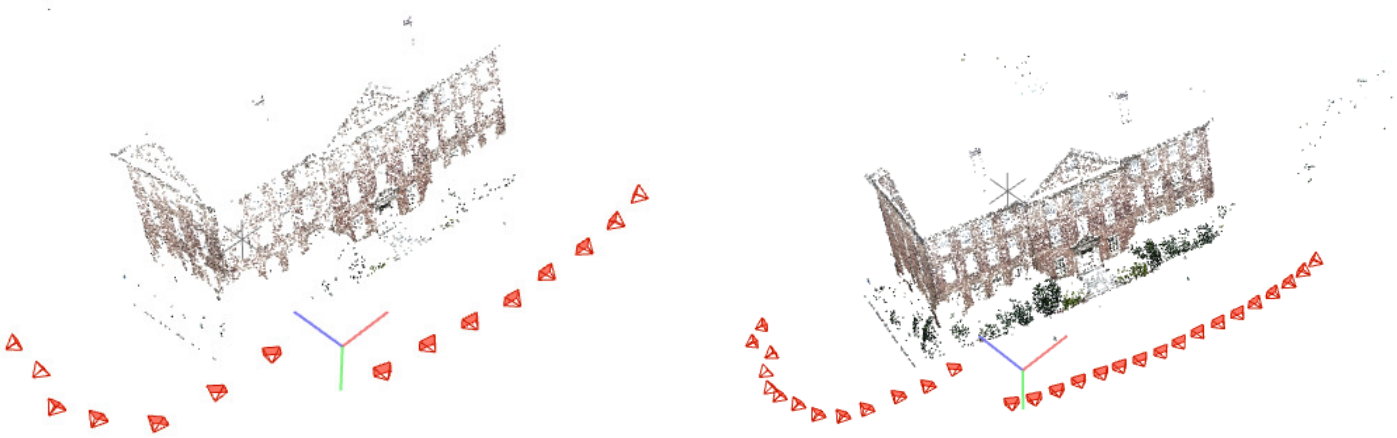
Table 1: COLMAP sparse-reconstruction results.

| Img | Mat | Reg | Points | Obs. | M Track | M Obs./Img | Reproj. Err. | F. Ext. | F. Match | Recon. T |
|---|---|---|---|---|---|---|---|---|---|---|
| 15 | Seq | 15/15 | 9,160 | 34,818 | 3.80 | 2321.2 | 0.31 | 0.012 | 0.009 | 0.082 |
| 15 | Exh | 15/15 | 9,467 | 35,739 | 3.77511 | 2382.6 | 0.31376 | 0.012 | 0.014 | 0.101 |
| 30 | Seq | 30/30 | 23,485 | 120,284 | 5.12174 | 4009.47 | 0.333783 | 0.021 | 0.019 | 0.230 |
| 30 | Exh | 30/30 | 24,947 | 126,067 | 5.05339 | 4202.23 | 0.345988 | 0.020 | 0.042 | 0.284 |

*Note:* **Img**: Images; **Mat**: Matching method (Seq: Sequential, Exh: Exhaustive); **Reg**: Registered images; **Obs.**: Observations; **M Track**: Mean track length; **M Obs./Img**: Mean observations per image; **Reproj. Err.**: Reprojection error; **F. Ext.**: Feature extraction time (min); **F. Match**: Feature matching time (min); **Recon. T**: Reconstruction time (min).

Increasing the dataset from 15 to 30 images greatly increased the number of reconstructed points and observations. Exhaustive matching produced more points than sequential matching for both dataset sizes: 9,472 versus 9,160 points for the 15-image set and 24,947 versus 23,485 points for the 30-image set. Sequential matching produced a slightly smaller mean reprojection error in both comparisons. This illustrates that reprojection error alone does not measure reconstruction completeness; the number of registered cameras, reconstructed points, observations, and visual coherence must also be considered.

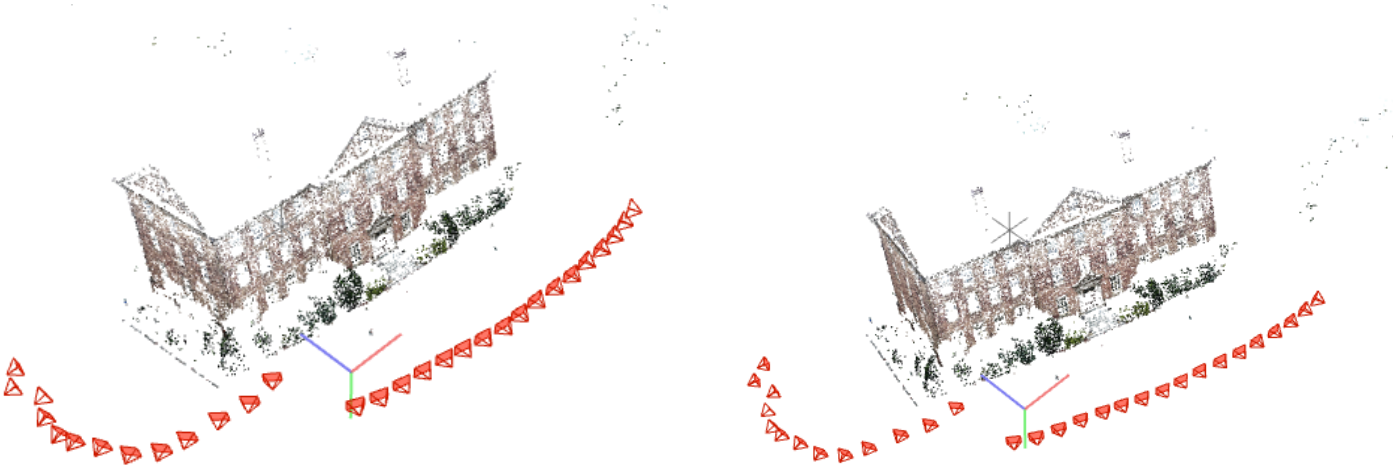
## 2.2 Qualitative evaluation



(a) COLMAP Image 15 Exhaustive

(b) COLMAP Image 30 Exhaustive

Figure 1: Comparison for COLMAP Image 15 vs 30.



(a) COLMAP Image 30 Sequential

(b) COLMAP Image 30 Exhaustive

Figure 2: Comparison for COLMAP Image 30 Sequential vs Exhaustive.

As shown in the quantitative results, Exhaustive matching successfully reconstructed a denser set of 3D points and more observations than Sequential matching across both dataset sizes (e.g., 24,947 points and 126,067 observations vs. 23,485 points and 120,284 observations for the 30-image set). Visualizing the sparse point clouds (see Figure 1), the 30-image dataset with exhaustive matching clearly yields a much more complete reconstruction of the building's facade and a smoother camera trajectory compared to the 15-image dataset. While Sequential matching produced a slightly lower mean reprojection error (e.g., 0.333 vs. 0.345 for the 30-image set), the visual completeness of the Exhaustive matching point cloud confirms that reprojection error alone is insufficient to evaluate overall reconstruction quality.

# 3 Gaussian Splatting

## 3.1 Training and evaluation protocol

The input scene contained all 30 photographs and the selected COLMAP reconstruction. The file `sparse/0/test.txt` explicitly assigned the three held-out filenames to the test split. The remaining 27 cameras were used for optimization. Training was performed once, and the model was evaluated at iterations 2000 and 6000.
The principal training command was:

```
!python train.py
-s /content/gaussian-splatting/south_building
-m /content/output/south_building_model
--iterations 6000
--save_iterations 2000 6000
--checkpoint_iterations 2000 6000
--eval
```

The saved GS models are the files `iteration_2000/point_cloud.ply` and `iteration_6000/point_cloud.ply`. Rendering uses a trained point cloud together with a camera pose and camera intrinsics to produce an image.

## 3.2 Quantitative results

Table 2: Gaussian Splatting rendering quality. Values are averages over each split.

| Iteration | Split | Views | PSNR (dB) | SSIM |
|---|---|---|---|---|
| 2000 | Training | 27 | 23.410 | 0.68033 |
| 2000 | Test | 3 | 21.805 | 0.66005 |
| 6000 | Training | 27 | 26.689 | 0.82973 |
| 6000 | Test | 3 | 22.546 | 0.76605 |

Between iterations 2000 and 6000, training PSNR increased by 3.279 dB and test PSNR increased by 0.741 dB. Training SSIM increased by 0.14940, while test SSIM increased by 0.10600. Thus, additional optimization improved both splits, although the larger training-set PSNR improvement indicates that training-view fidelity increased more strongly than held-out-view fidelity.

## 3.3 Visual comparison

Figures 3 and 4 compare each held-out photograph with the corresponding rendering.




Figure 3: Held-out ground-truth photographs (left) and GS renderings (right) after 2000 iterations.




Figure 4: Held-out ground-truth photographs (left) and GS renderings (right) after 6000 iterations.

Increasing iterations from 2,000 to 6,000 visibly sharpens the brickwork and window mullions, yet the vegetation remains unstable and blurred compared to the facade. Reconstruction artifacts persist along the image boundaries, foliage edges, and

the roof cupola due to sparser camera coverage and extreme viewing angles. Among the test views, the frontal perspective (P1180208) reconstructs cleanly due to dense viewpoint coverage and low occlusion, while the oblique view (P1180198) exhibits peripheral degradation at grazing angles. The low-angle view (P1180218) proves the most challenging, suffering from cupola occlusion, intrusive foreground foliage, and transient pedestrians whose movement breaks the static scene assumption, resulting in translucent ghosting artifacts.

### 3.4 Training and rendering time

Report these times separately from the COLMAP times in Table **??**.

Table 3: Gaussian Splatting elapsed times.

| Stage | Elapsed time |
|---|---|
| Training to 6000 iterations | training time: 1331.7 seconds (22.19 minutes) |
| Rendering iteration 2000 (27 train + 3 test views) | 13.979 s |
| Rendering iteration 6000 (27 train + 3 test views) | 13.763 s |

### 3.5 Evaluation limitation

The three test photographs were excluded from GS optimization, but they were not excluded from the preceding COLMAP reconstruction. Their camera poses were estimated from those photographs, and their feature observations could contribute to the initial sparse point cloud. Consequently, the experiment measures rendering at held-out RGB views but is not a completely independent geometric test.

## A Relevant Source Code

The explicit split was implemented by reading one held-out filename per line from `sparse/0/test.txt`, matching those names to the COLMAP cameras, and assigning the remaining cameras to the training set. The essential logic was:

```
    if eval:
        test_list_path = os.path.join(path, "sparse/0", "test.txt")
        if os.path.exists(test_list_path):
            with open(test_list_path, "r", encoding="utf-8") as test_file:
                test_filenames = {
                    line.strip() for line in test_file
                    if line.strip() and not line.lstrip().startswith("#")
                }
                test_names = {
                    os.path.splitext(os.path.basename(name))[0]
                    for name in test_filenames
                }

            available_names = {c.image_name for c in cam_infos}
            missing_names = test_names - available_names
            if missing_names:
                raise ValueError(
                    f"Test images are absent from the COLMAP model: {sorted(missing_names)}"
                )

            train_cam_infos = [c for c in cam_infos if c.image_name not in test_names]
            test_cam_infos = [c for c in cam_infos if c.image_name in test_names]
        elif llffhold > 0:
            train_cam_infos = [c for idx, c in enumerate(cam_infos) if idx % llffhold != 0]
            test_cam_infos = [c for idx, c in enumerate(cam_infos) if idx % llffhold == 0]
        else:
            raise ValueError(
                "Evaluation requires sparse/0/test.txt when llffhold is 0. "
                "Put one held-out image filename on each line."
            )
```

PSNR and SSIM were computed independently for every rendered training and test image and then averaged within each split. Tensors were made contiguous before evaluation because the reference PSNR implementation uses a tensor `view` operation.

```python
# Compute PSNR and SSIM for every train/test render directory.
# This avoids metrics.py's test-only aggregation and prints values needed in the report.
from PIL import Image
from torchvision.transforms.functional import pil_to_tensor
from utils.image_utils import psnr
from utils.loss_utils import ssim

rows = []
for iteration in (2000, 6000):
    for split in ("train", "test"):
        result_dir = MODEL_ROOT / split / f"ours_{iteration}"
        render_dir, gt_dir = result_dir / "renders", result_dir / "gt"
        psnr_values, ssim_values = [], []
        for render_path in sorted(render_dir.glob("*.png")):
            gt_path = gt_dir / render_path.name
            render_tensor = (pil_to_tensor(Image.open(render_path).convert("RGB")).float().cuda() / 255.0).
                unsqueeze(0).contiguous()
            gt_tensor = (pil_to_tensor(Image.open(gt_path).convert("RGB")).float().cuda() / 255.0).unsqueeze(0).
                contiguous()
            psnr_values.append(psnr(render_tensor, gt_tensor).mean().item())
            ssim_values.append(ssim(render_tensor, gt_tensor).item())
        rows.append((iteration, split, len(psnr_values), sum(psnr_values)/len(psnr_values), sum(ssim_values)/len
            (ssim_values)))

print("iteration  split  views  PSNR(dB)  SSIM")
for iteration, split, count, mean_psnr, mean_ssim in rows:
    print(f"{iteration:9d}  {split:5s}  {count:5d}  {mean_psnr:8.3f}  {mean_ssim:.5f}")
```